

Zero Trust SDN প্রজেক্ট — Detail Study Guide

বোঝার জন্য + Mentor-কে বোঝানোর জন্য (সবকিছু আসল কোড থেকে যাচাই করা)

১. এক নজরে পুরো সিস্টেম (Architecture)

Request flow (লেয়ার ডায়াগ্রাম):

App (h1..h4) --HTTP POST--> /zt/register (একবার — টোকেন পাওয়ার জন্য)
App --HTTP POST--> /zt/request (প্রতিটা কাজের আগে)
→ handle_northbound_request() (ryu_controller.py:184)
→ PEP.enforce() (policy_enforcement_point.py:57)
→ TrustVerificationEngine.verify_request() — ৭টা লেয়ার
→ PolicyEngine.evaluate_request() — policy match
→ **allow** হলে: OpenFlow flow_mod switch-এ বসে (real change)
→ **deny** হলে: HTTP **403**, switch-এ হাতও দেওয়া হয় না
→ Trust update: $composite = 0.7 \times session + 0.3 \times reputation$
→ Statistics (latency, allow/deny count)

Module গুলো ও এদের দায়িত্ব

File	Class	মূল কাজ
<code>ryu_controller.py</code>	<code>ZeroTrustController</code> + <code>ZeroTrustRestController</code>	RYU app; OpenFlow 1.3; L2 learning switch; WSGI REST API (port 8080); সব northbound request Zero Trust pipeline দিয়ে; allow হলেই OpenFlow action বাস্তবে বসে (<code>_apply_openflow_action</code> , line 241); stats API
<code>trust_verification.py</code>	<code>TrustVerificationEngine</code>	৭-লেয়ার verification pipeline + app registration + token generation + trust score store
<code>policy_engine.py</code>	<code>PolicyEngine</code>	verification-এর পর policy match করে allow/deny সিদ্ধান্ত (<code>config/policies.json</code>)
<code>policy_enforcement_point.py</code>	<code>PolicyEnforcementPoint</code>	Fail-closed gatekeeper; verdict allowed না হলে deny; counters + audit log; flow rule install/drop
<code>trust_scoring.py</code>	<code>TrustScoring</code>	দীর্ঘমেয়াদি reputation: allow +0.1, critical fail -0.3, RBAC/ABAC fail -0.1, decay -0.01/মিনিট
<code>baseline_controller.py</code>	control group	Zero Trust ছাড়া একই REST surface — তুলনার জন্য (paper-এ "baseline")

২. রেজিস্ট্রেশন (একবার) — কী হয়

POST `/zt/register` body: `{app_id, api_key, role, device_id, device_fingerprint, hostname}`

- `register_application()` → `api_key` store; `trust_score` = **0.5** (neutral); device registry-তে ঢোকে
- Server-side `generate_token()`: `HMAC-SHA256(secret, "app_id:timestamp")` — **৬০ সেকেন্ডে expire** (`expires_in: 60`)
- Token প্রতিটা পরের request-এ দিতে হয় — replay ঠেকাতে short-lived

৩. ৭টা Verification Layer — বিস্তারিত

প্রতিটা request-এ `verify_request()` (`trust_verification.py:132`) এই ৭ ধাপ চালায় — কোনো ধাপ fail = immediate deny:

#	Layer (method)	কী check হয়	Fail-এ কী হয়
১	Token (<code>_verify_token</code> :220)	token আছে? ৬০s-এর ভেতরে? <code>HMAC-SHA256(secret, app_id:timestamp)</code> মিলে?	"Token verification failed", score 0.0 — সবচেয়ে কঠিন block
২	API Key (<code>_validate_api_key</code> :253)	রেজিস্ট্রেশনের api_key-এর সাথে মিলে?	score 0.1
৩	Device (<code>_verify_device_identity</code> :271)	device_id registry-তে? fingerprint মিলে? blacklist-এ নেই?	score 0.2
৪	RBAC (<code>_check_rbac</code> :299)	role-এর permission list-এ action আছে?	score 0.3
৫	ABAC (<code>_check_abac</code> :318)	attribute policies (policy_001–005) match? (trust_score_min, time_range, token_valid, trusted_device)	score 0.4
৬	Behavioral (<code>_analyze_behavior</code> :372)	গত ৩s-এ >৬ request? প্রথমবার resource? নতুন app-এর delete?	"Behavioral anomaly", score = 1.0 – burst_penalty
৭	Trust scoring (<code>_calculate_trust_score</code> :447)	weighted sum → 0.6-এর কম হলে deny	"Trust score below 0.6"

RBAC role → permission (code থেকে)

Role	Permissions
admin	read, write, delete, configure, monitor
network_operator	read, write, monitor
monitoring_app	read, monitor
guest_app	read

Trust score formula (Step 7 weighted sum)

Step	Weight	Score
token_verification	0.20	1.0 (valid) / 0.0
api_key_validation	0.15	1.0 / 0.0
device_identity	0.15	1.0 / 0.0
rbac_check	0.20	1.0 / 0.0
abac_check	0.15	1.0 / 0.0
behavioral_analysis	0.15	1.0 - anomaly_score

Threshold: 0.6 — সব pass করলে ~1.0, তাই legit-রা সহজে allow পায়।

৪. Behavioral Analysis — Flood attack কীভাবে ধরা হয়

- **Burst check:** গত ৩ সেকেন্ডে **>৬টা** request = anomalous burst (আগে ছিল 10-in-5s, tighten করা হয়েছে)
- **Progressive burst penalty:** `burst_penalty = min(1.0, 0.1 × (burst_size - 6))` — 7th request থেকে প্রতি অতিরিক্ত request-এ -0.1 → flood করলে score দ্রুত **০-তে** যায়
- Demo-তে তাই flood attack-এ **৬টা allowed** (নিশ্চিত হওয়ার জন্য) → তারপর block
- অন্যান্য anomaly: প্রথমবার resource access (+0.1), নতুন app-এর delete attempt (+0.2)

৫. Continuous Trust Scoring + Hybrid Model (নতুনত্ব)

দুই স্তরের trust:

- **Session trust (S):** এই request-এর ৭-লেক্সার verdict-এর score (0–1)
- **Reputation (R):** `TrustScoring` module-এ জমা দীর্ঘমেয়াদি ইতিহাস — allow +0.1; auth/identity/behavioral fail -0.3; RBAC/ABAC fail -0.1 (কম গুরুতর); প্রতিমিনিটে -0.01 decay
- **Composite:** `C = 0.7×S + 0.3×R` (ryu_controller.py:218) — API-তে এই score-ই দেখায়

কেন tiered penalty: legit monitor-এর মতো অ্যাপ মাঝেমধ্যে policy probe করে (যেমন write চেষ্টা)। সেটা -0.3 হলে reputation নষ্ট হয়ে legit-ও lock হয়ে যেত (আগে এমন bug ছিল!)। তাই RBAC/ABAC fail = হালকা -0.1, কিন্তু credential/behavioral fail = গুরুতর -0.3।

৬. PEP — Fail-Closed Enforcement

- `enforce()` (policy_enforcement_point.py:57): প্রথমে trust verdict নেয়; **allowed না হলে সাথে সাথে deny** — policy evaluation-ও শুরু হয় না। "fail closed" = অনিশ্চিত হলে deny।
- Verdict-এ `trust_score`, `reason`, `matched_policy`, `verification_steps` সব response-এ যায় (attribution — audit-এর জন্য)
- Deny হলে switch-এ কোনো OpenFlow message যায় না; allow হলেই `_apply_openflow_action()`: `install_flow` (forward) বা `drop_flow` (DROP rule, priority 100)

- Counters: allowed/blocked/total + block_rate; সব enforcement audit log-এ

৭. আমরা যে ৫টা Bug ধরেছি ও ঠিক করেছি (মেন্টর-এর কাছে highlight!)

#	Bug	আগে (ভুল)	পরে (ঠিক)	কেন গুরুত্বপূর্ণ
১	PEP allowed check নাই	Trust verdict deny করলেও request চলত (behavioral attack কখনো block হতো না)	Fail-closed early-deny — verdict allowed না হলে immediate 403	ফ্রেমওয়ার্কের মূল promise ("attack আটকাবে") ভেঙে যাচ্ছিল
২	Verdict থেকে trust_score/reason হারাত	সব response-এ trust=0.0 দেখাত	Attribution: trust_score, reason, matched_policy, verification_steps	stats/audit সঠিক হয়; paper-এর trust graphs বাস্তব হয়
৩	সব failure-এ -0.3	legit monitor RBAC probe-এ reputation নষ্ট → lock	Tiered: auth/identity/behavioral -0.3; RBAC/ABAC -0.1	Legit অ্যাপ ঠিক থাকে (FPR 0.000-এর কারণ)
৪	Flood threshold আলগা	10-in-5s — flood অনেকক্ষণ pass	6-in-3s + progressive burst penalty	Flood দ্রুত ধরা পড়ে, trust 0-তে যায়
৫	Client token timestamp bug	রেজিস্ট্রেশন token-এ fresh timestamp পাঠাত → token মাঝে মাঝে fail (second boundary)	<code>self.token_timestamp</code> — token issue-এর সময় ব্যবহার	Legit-দের মাঝেমধ্যে ভুল deny হওয়া বন্ধ

৮. Evaluation Methodology — কীভাবে প্রমাণ করলাম

Setup

- Ubuntu 22.04 VM (VirtualBox), Ryu 4.34 + Mininet, OpenFlow 1.3, 1 OVS switch + h1..h4
- Baseline:** `baseline_controller.py` — একই REST surface, কিন্তু কোনো verification নেই (control group)
- Scripts: `tools/bench_phase_b.py` (B1-B3), `tools/bench_scalability.py` (B4) — VM-এ run, JSON output

B1 — Detection battery

- ৫টা attack class × **50 runs**: no_credentials, fake_device, replay, unauthorized, flood
- Metric: `detection_rate = denied / total`; flood-এর জন্য আলাদা `request_block_rate`
- False positive test: legit monitor + admin কাজ × 50 runs → কতবার ভুলভাবে deny হয়

B2 — Latency (overhead)

- Sequential:** 500 request (mix of legit + attacks) — baseline বনাম ZT-তে একই workload
- Concurrent:** 100 request, 10 threads — controller-এর eventlet GIL-এর প্রভাব মাপতে
- Metrics: avg, p95, max

B3 — Trust evolution trace

- ৩টা অ্যাপের trust score per-request log: admin (সং), monitor (RBAC probe-সহ), compromised_app (flood)
- দেখায়: সং-দের trust বাড়ে, compromised-এর ০-তে নামে, monitor dip করে self-heal করে

B4 — Scalability

- Star topology: 4 → 8 → 16 hosts; controller restart ছাড়া আকার বাড়ানো
- Metrics: ping loss %, sequential avg/p95, concurrent avg — latency flat থাকলে pipeline cost topology-র সাথে বাড়ে না

৯. Results (আসল পরিমাপ — সব numbers মুখস্থ রাখার মতো)

B1 — Detection & FPR (50 runs/class)

Attack class	Detection rate	মন্তব্য
no_credentials	1.000	Token fail → 403
fake_device	1.000	Device identity fail → 403
replay	1.000	Token signature/timestamp fail → 403
unauthorized	1.000	RBAC fail → 403
flood	1.000 (detect)	request_block_rate 0.600 — ৬টা tolerated, 7th থেকে block

Legit app	False positive rate	মন্তব্য
legitimate_monitor	0.000 (0/50)	কখনো ভুল deny হয়নি
legitimate_admin	0.000 (0/50)	কখনো ভুল deny হয়নি

B2 — Latency (baseline বনাম ZT)

Metric	Baseline	Zero Trust	Overhead
Sequential avg	4.038 ms	5.520 ms	+1.48 ms (+37%)
Sequential p95	7.352 ms	9.078 ms	+1.73 ms
Concurrent avg (10 workers)	25.158 ms	50.181 ms	~2× (eventlet GIL)

Concurrent-এ 2× আসলেও absolute মান millisecond-স্তরের; discussion-এ transparently বলা আছে।

B3 — Trust evolution (হাইলাইট)

- legitimate_admin: 0.88 → **1.00** (সব allow, trust বাড়তে থাকে)
- legitimate_monitor: RBAC probe-তে dip (~0.3–0.45) → আবার **self-heal**
- compromised_app (flood): 0.88 → **0.00** (progressive lockout)

B4 — Scalability (4/8/16 hosts)

Hosts	Ping loss	Seq avg	Seq p95	Conc avg
4	0.0%	3.612 ms	5.204 ms	16.733 ms
8	0.0%	3.804 ms	4.869 ms	27.295 ms
16	0.0%	3.660 ms	4.905 ms	26.768 ms

Interpretation: verification cost প্রতিটা request-এ একই — topology বড় হলেও latency flat।

১০. Mentor-এর সম্ভাব্য প্রশ্ন + উত্তর (প্রস্তুতি)

Q: Zero Trust-এর মূল নীতি কী?

A: "Never trust, always verify" — প্রতিটা request, প্রতিবার verify। Network-এ ঢুকলেই trust নয় (NIST SP 800-207)।

Q: কেন SDN-এ দরকার?

A: Northbound API-তে আগে কোনো auth ছিল না — install করা app-ই controller-এর মতো ক্ষমতা পেত। Compromised/fake app নেটওয়ার্ক বদলে দিতে পারত।

Q: Trust score কীভাবে হিসাব হয়?

A: ৬টা weighted step (0.20/0.15/0.15/0.20/0.15/0.15) যোগ → 0.6 threshold। আর reputation আলাদা module-এ (+0.1/-0.3/-0.1/decay)। Final = 0.7×session + 0.3×reputation।

Q: Flood attack ধরার logic?

A: ৩s window-এ >৬ request = burst; penalty $0.1 \times (n-6) \rightarrow$ trust ০-তে নামে; 7th request থেকে block।

Q: False positive ০ কীভাবে নিশ্চিত?

A: Tiered penalty + fail-closed-এ token/identity check-ই deny করে, legit-দের সব check pass → +0.1 করে trust বাড়ে। 50-run test-এ ০/50।

Q: Overhead কত?

A: Sequential-এ +37% (1.48 ms) — sub-millisecond থেকে millisecond-স্তরে। Concurrency-তে eventlet GIL-এর কারণে 2× — limitation হিসেবে স্বীকার।

Q: Existing কাজের সাথে পার্থক্য?

A: FortNox/Rosemary/PermOF authentication/containment দেয়, কিন্তু continuous verification + dynamic policy + behavioral trust scoring একসাথে নেই — এটাই gap (Aliyu 2020-র extension)।

Q: Simulation-এর সীমাবদ্ধতা?

A: Mininet emulation, single switch, eventlet GIL, secret keys code-এ হার্ডকোড করা — production-এ

১১. কোড কীভাবে চালাতে হয় (মেন্টর-কে live দেখাতে)

- 1. `sudo bash setup/install_dependencies.sh`
- 2. Terminal A: `sudo ryu-manager controller/ryu_controller.py`
- 3. Terminal B: `sudo python3 topology/network_topology.py`
- 4. Mininet CLI-তে: `h1 python3 ../apps/http_app_client.py --app monitor` (আর admin, attacker attacks)
- 5. Check: `curl http://127.0.0.1:8080/zt/stats` এবং `curl http://127.0.0.1:8080/zt/trust/legitimate_admin`

১২. বাকি কাজের Checklist

#	কাজ	স্ট্যাটাস
১	Paper-এ author name + affiliation	বাকি
২	<code>paper.tex</code> Overleaf-এ compile → PDF	বাকি
৩	Paper-এর ভাষা নিজের করে rewrite (AI-detector সতর্কতা)	বাকি
৪	Demo screen recording (Win+G) — mentor-কে	বাকি
৫	GitHub collaborator invite	বাকি
৬	Target venue বাছাই (IEEE Access / conference)	বাকি
৭	Viva prep — এই guide পড়ে যাওয়া	চলছে 😊

Detail study guide — কোড line reference সহ (ryu_controller.py, trust_verification.py, trust_scoring.py, policy_enforcement_point.py থেকে যাচাই করা)।